

When the program starts, class Loader loads the code , to code segment and the JVM starts executing from main method

→At compile time the main time is stored in static segment but the at execution time activation record is created at **stack segment**

→? When the **main() method starts executing**, a **stack frame (activation record)** is created in the **Stack** for the main() method.

? →When we create an object using the **new keyword**, memory for that object is allocated at **runtime in the Heap**.

=>new keyword give memory address to reference s1

→Instance variables are part of the object and receive default values

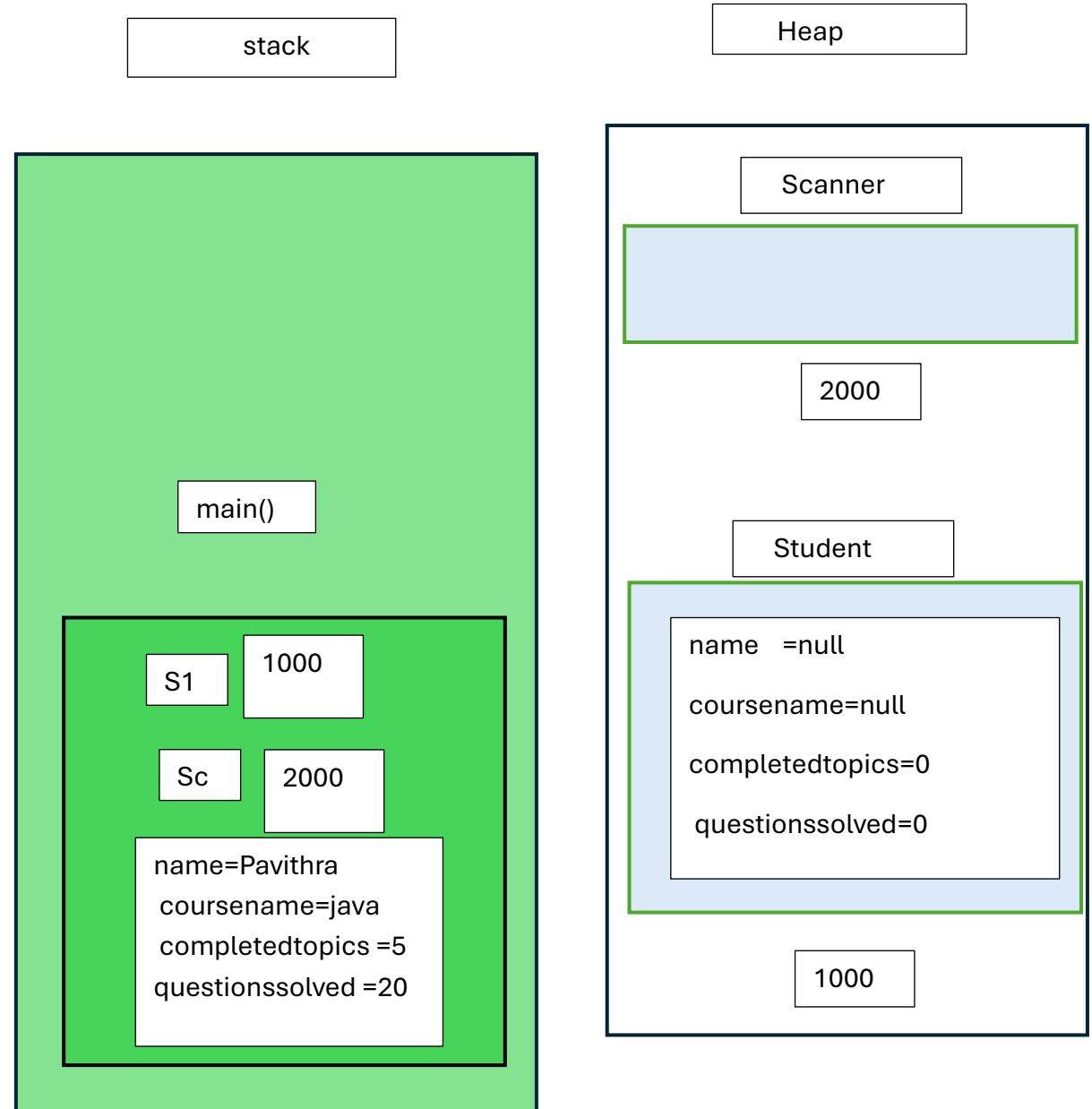
→A Scanner object is also created in the Heap, and its reference variable sc

→In the main() method, we use the **sc reference** to take input from the user.
Name

coursename

completedtopics

questionssolved



➔ When setDetails() is called, a **new stack frame (activation record)** is created for the setDetails() method.

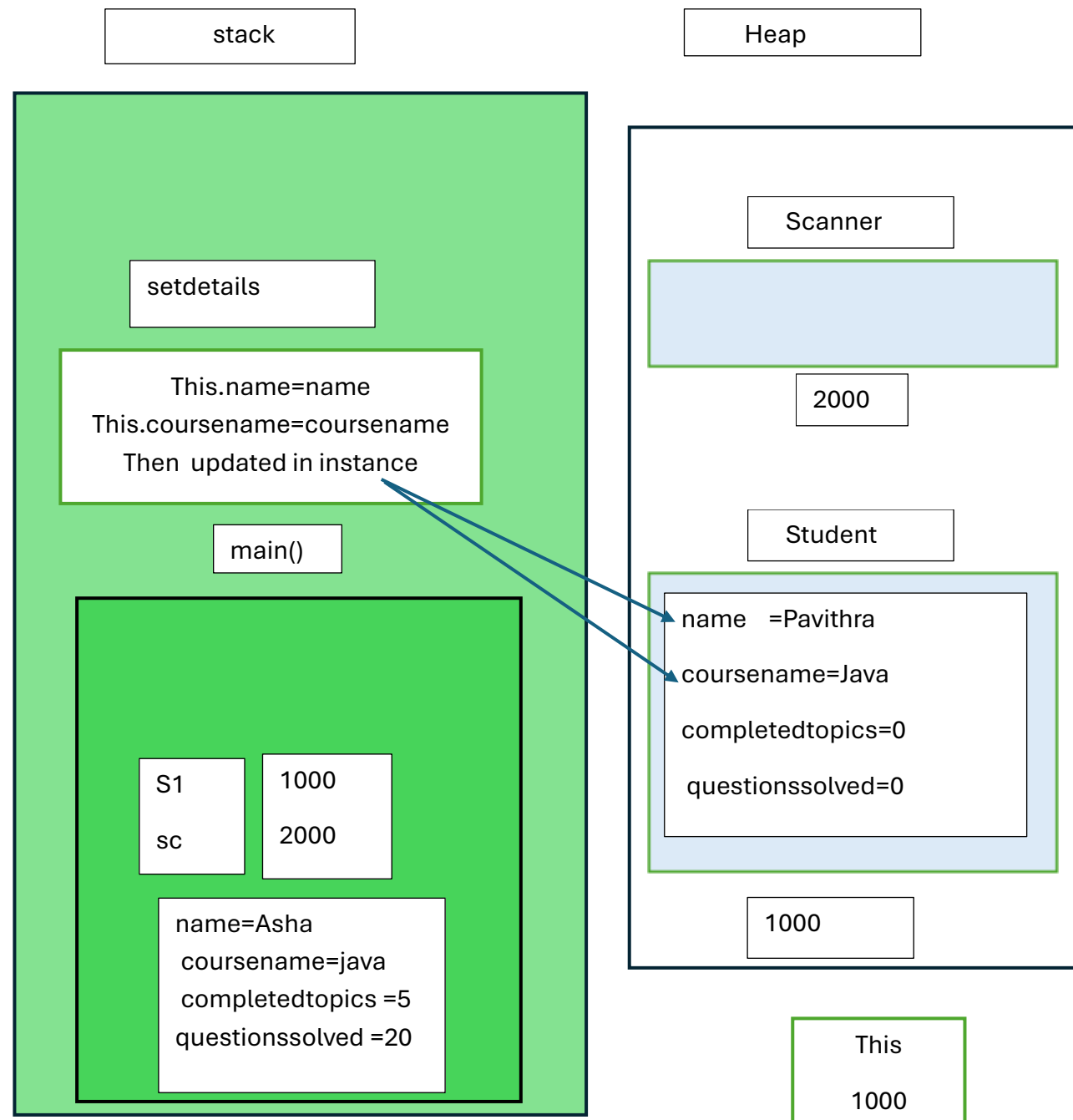
? The values of name and courseName are passed as **parameters** to the setDetails() method.

? Inside the method, we use the **this keyword** to refer to the current object (s1).

➔ The this keyword helps us **distinguish the instance variables from the parameters** when they have the same names.

➔? The setDetails() method sets the values of name and courseName in the **instance variables of the s1 object**.

? After the setDetails() method finishes execution and returns, its **stack frame is removed from the Stack**.



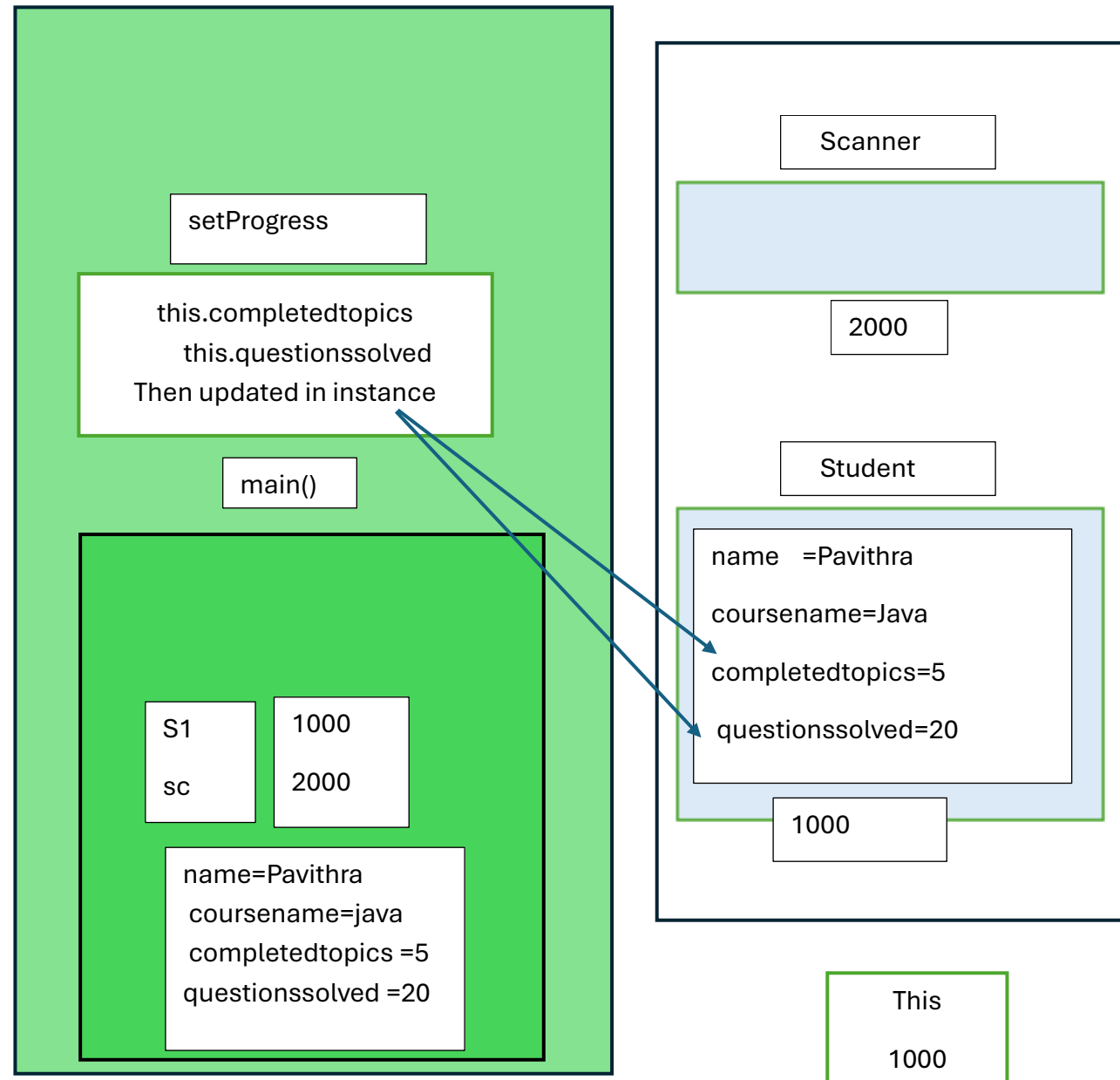
When `setProgress()` is called, a **new stack frame (activation record)** is created for the `setProgress()` method.

The values of `completedTopics` and `questionsSolved` are passed to the method as **parameters**.

➔ The `this` keyword helps us distinguish between the **instance variables** and **parameters** when they have the same names.

➔ ? The `setProgress()` method updates the **instance variables of the s1 object**.

? After the `setProgress()` method completes execution, its **stack frame is removed from the Stack**.



displayDetails()

→ ? The displayDetails() method starts executing. A **new stack frame (activation record)** is created for displayDetails().

→ The method prints the details stored in the s1 object. After the **control leaves the displayDetails() method**, the method finishes execution. Its **activation record (stack frame) is removed from the Stack**. The s1 object in the **Heap is not removed**

displayProgress()

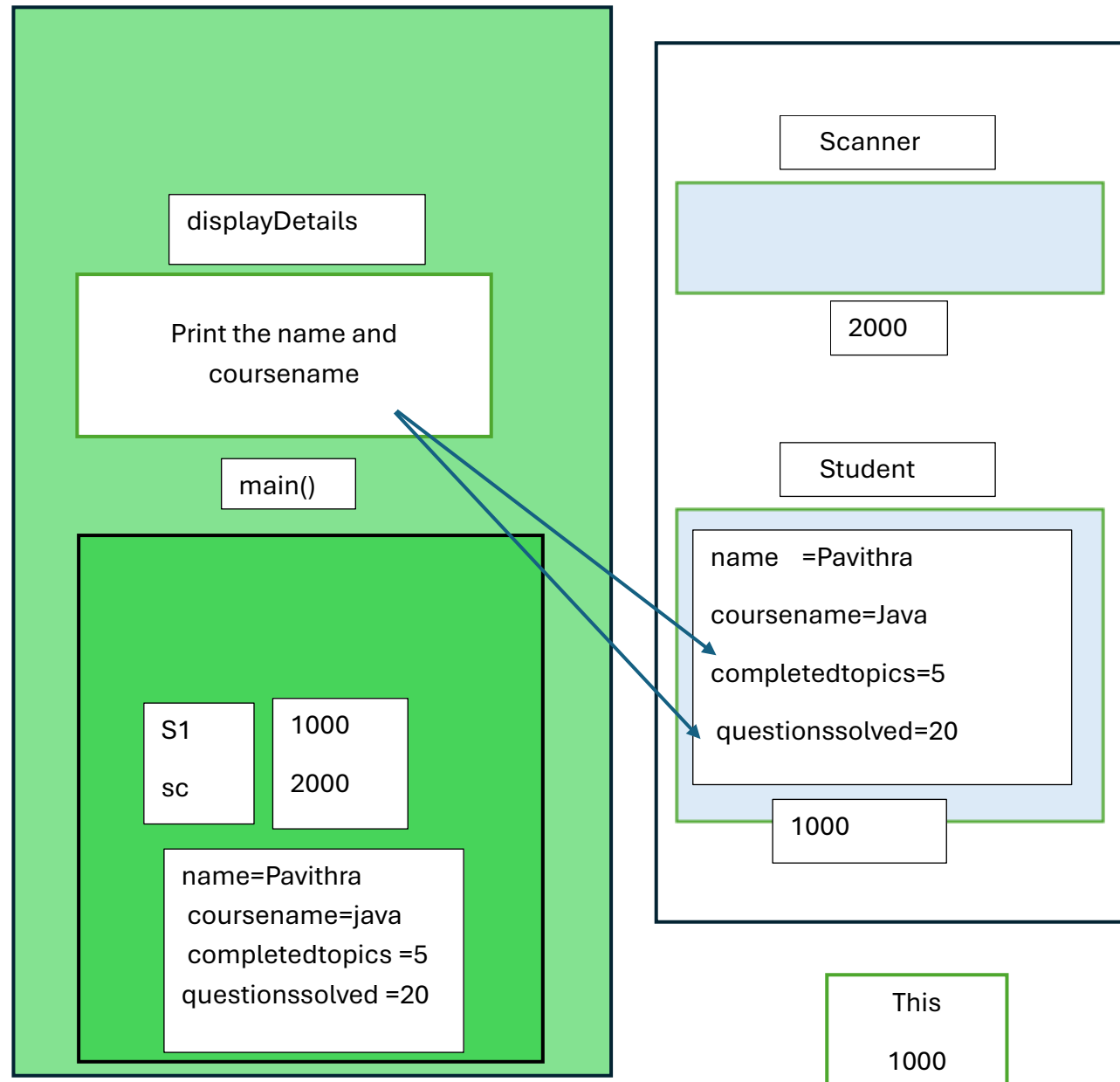
→ ? A **new stack frame (activation record)** is created for displayProgress().

? The method prints the progress details of the s1 object.

→ ? The method prints the required statements.

? When the **control leaves the displayProgress() method**, the method finishes.

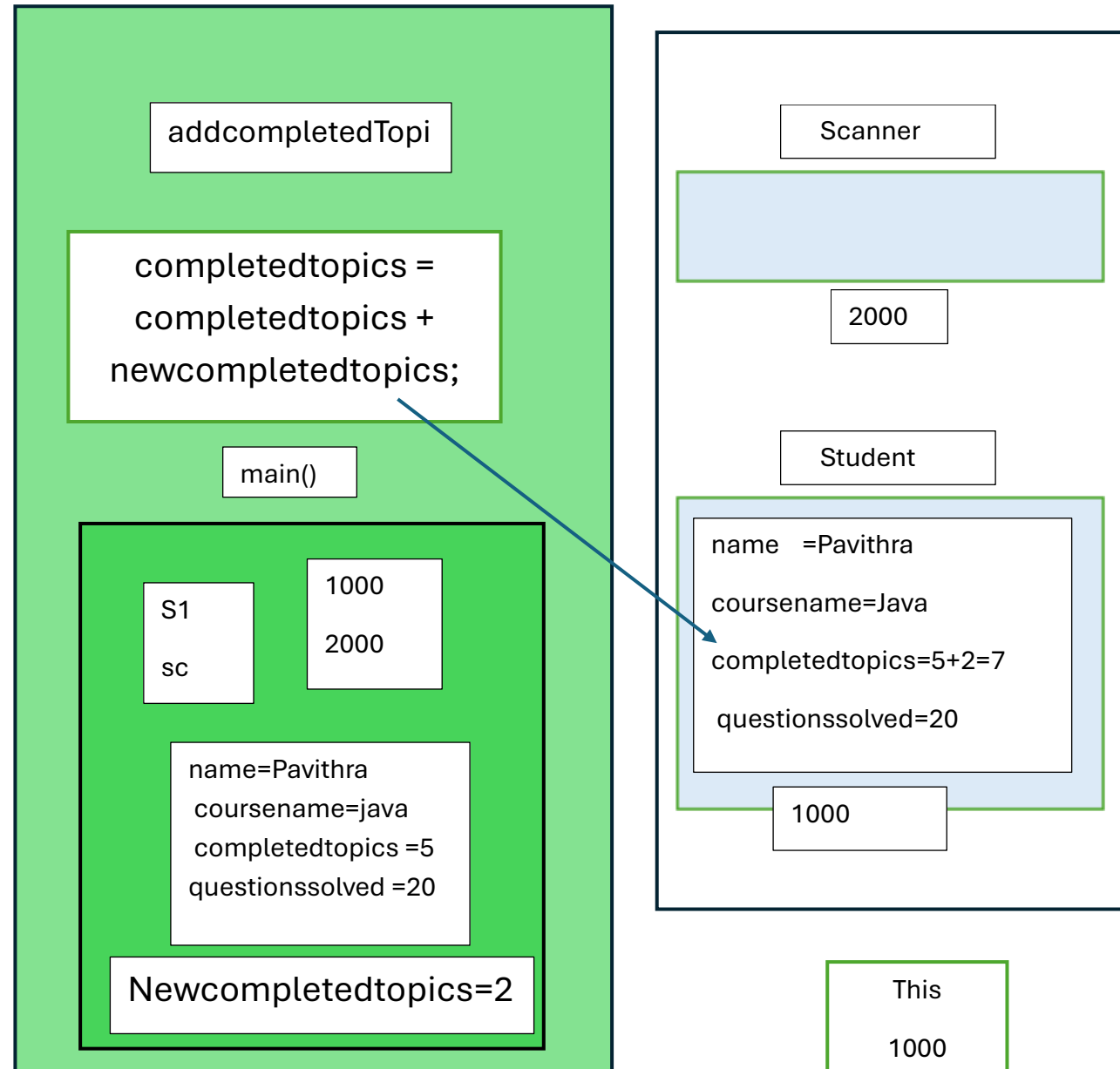
? Its **activation record is removed from the Stack**.



```
int addcompletedTopics(int  
newcompletedtopics) {
```

➔ newcompletedtopics are added to
completed topics and thus
completedtopics are updated

➔ the activation record is deleted
after the ctrl leaves the method



```
int addquestionsSolved(int  
newquestionssolved)
```

➔ newquestionssolved; are added
to completed topics and thus
questionssolved are updated

➔ the activation record is deleted
after the ctrl leaves the method

